Reti logiche

Appunti

Marco Cerbella

Università degli Studi di Perugia – A.A. 2026/2027
Aggiornato al 9 novembre 2026

Indice

1 Algebra di Boole e algebra di commutazione (Lezione 1 – 21 settembre 2026) 2

2 Funzioni ed espressioni booleane (Lezione 2 – 6 ottobre 2026) 4

3 Porte logiche e circuiti logici (Lezione 3) 10

4 Progettazione di reti logiche combinatorie (Lezione 4) 13

5 Ottimizzazione: mappe di Karnaugh (Lezione 5) 15

6 Mappe di Karnaugh: il punto di vista del libro (Lezione 6) 17

7 Reti sequenziali e macchine a stati finiti (Lezione 7) 20

8 Bistabili, latch, flip-flop e sintesi di circuiti sequenziali (Lezione 8) 24

1 Algebra di Boole e algebra di commutazione

Lezione 1 – 21 settembre 2026

Cenno storico

L'algebra di Boole fu introdotta da George Boole (1815–1864) nell'Ottocento per dimostrare affermazioni logiche svolgendo calcoli algebrici: per Boole la logica va associata alla matematica e non alla filosofia. Nel 1936 Claude Shannon (1916–2001), studiando al MIT un circuito di oltre cento relè collegato all'analizzatore differenziale di Bush, intuì che l'algebra di Boole è un modello formale per l'*analisi* e la *sintesi* di tali sistemi: due interruttori in *serie* (*parallelo*) si descrivono con AND (OR), mentre il NOT si realizza con il contatto posteriore di un relè. Dalla sua tesi (1938) nasce l'algebra di commutazione, base delle reti logiche.

1.1 Algebra di Boole

Definizione. Un'*algebra di Boole* è una struttura algebrica

$$\mathcal{B} = \langle A, +, \cdot, ', 0, 1 \rangle$$

dove:

- A è l'insieme degli elementi su cui l'algebra opera (*supporto*);
- $+$ è l'operatore di disgiunzione (OR);
- $\cdot$ è l'operatore di congiunzione (AND);
- $'$ è l'operatore unario di complementazione (NOT);
- 0 è l'elemento neutro rispetto a $+$, 1 è l'elemento neutro rispetto a $\cdot$;

e valgono i seguenti assiomi (*postulati di Huntington*).

1.1.1 Postulati di Huntington

1. **Commutatività**: $\forall a, b \in A$: $a + b = b + a$ e $a \cdot b = b \cdot a$.
2. **Elementi neutri**: esistono 0 e 1 tali che, per ogni $a \in A$, $a + 0 = a$ e $a \cdot 1 = a$.
3. **Distributività** (di ciascuna operazione rispetto all'altra): $\forall a, b, c \in A$

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$
$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

4. **Complemento**: per ogni $a \in A$ esiste ed è unico a' tale che $a + a' = 1$ e $a \cdot a' = 0$.

Teorema (Principio di dualità). Ogni proposizione derivata dagli assiomi di Huntington resta valida scambiando ovunque tra loro le operazioni $+$ e $\cdot$ e gli elementi neutri 0 e 1.

Osservazione. In particolare se $\langle A, +, \cdot, ', 0, 1 \rangle$ è un'algebra di Boole, lo è anche $\langle A, \cdot, +, ', 1, 0 \rangle$.

1.1.2 Esempi

Esempio (Algebra delle classi)**.** Sia A un insieme e $\mathcal{P}(A)$ l'insieme delle sue parti (tutti i sottoinsiemi di A). La struttura $\langle \mathcal{P}(A), \cup, \cap, ^-, \emptyset, A\rangle$ è un'algebra di Boole:

- $\cup$ e $\cap$ sono commutative;
- $\emptyset$ è neutro rispetto a $\cup$ e A rispetto a $\cap$: per ogni $X \in \mathcal{P}(A)$, $X \cup \emptyset = X$ e $X \cap A = X$;
- distributività e unicità del complemento si verificano con i diagrammi di Venn.

È un'algebra di Boole *chiusa*: $\forall X, Y \in \mathcal{P}(A)$, $X \cap Y$, $X \cup Y$ e $\overline{X} = A \setminus X$ sono sottoinsiemi di A, quindi elementi di $\mathcal{P}(A)$.

Teorema (Rappresentazione di Stone). Per ogni algebra di Boole finita $\mathcal{A}$ esiste un insieme finito X tale che $\mathcal{A}$ è isomorfa a $\langle \mathcal{P}(X), \cup, \cap, ^-, \emptyset, X\rangle$.

Conseguenze:

- la cardinalità del supporto di un'algebra di Boole finita è sempre una potenza di 2;
- per dimostrare che le algebre di Boole finite godono di una proprietà basta dimostrarla per l'algebra delle classi (ad esempio con i diagrammi di Venn).

Esempio (Altre algebre di Boole)**.** $\langle \{0,1\}, \min, \max, 1-x, 0, 1\rangle$ è un'algebra di Boole; così come quella ottenuta scambiando le operazioni come nell'osservazione sopra.

1.1.3 Variabili booleane

Sia $\mathcal{A} = \langle A, +, \cdot, ', 0, 1\rangle$ un'algebra di Boole. Una *variabile booleana* è un simbolo che indica un qualsiasi elemento del supporto A; il suo valore è un elemento di A. Le variabili si indicano con le ultime lettere dell'alfabeto $(x, y, \dots)$, gli elementi del supporto con le prime $(a, b, \dots)$ e con i simboli 0 e 1.

1.1.4 Proprietà dell'algebra di Boole

Proprietà. Per ogni x, y, z valgono:

- **associatività**: $x + (y + z) = (x + y) + z$ e $x \cdot (y \cdot z) = (x \cdot y) \cdot z$;
- **idempotenza**: $x + x = x$ e $x \cdot x = x$;
- **elemento nullo**: $x + 1 = 1$ e $x \cdot 0 = 0$;
- **unicità del complemento**: x' è unico;
- **assorbimento**: $x + x \cdot y = x$ e $x \cdot (x + y) = x$;
- **semplificazione**: $x + x' \cdot y = x + y$ e $x \cdot (x' + y) = x \cdot y$;
- **involuzione**: $(x')' = x$;
- **leggi di De Morgan**: $(x + y)' = x' \cdot y'$ e $(x \cdot y)' = x' + y'$;
- **consenso**: $x \cdot y + x' \cdot z + y \cdot z = x \cdot y + x' \cdot z$ e $(x+y) \cdot (x'+z) \cdot (y+z) = (x+y) \cdot (x'+z)$.

Ciascuna si può verificare con i diagrammi di Venn, oppure dimostrare dagli assiomi.

Esempio (Dimostrazione dell'idempotenza)**.**

$$\begin{aligned} x \cdot x &= (x \cdot x) + 0 && \text{elemento neutro} \\ &= (x \cdot x) + (x \cdot x') && \text{complemento} \\ &= x \cdot (x + x') && \text{distributiva} \\ &= x \cdot 1 && \text{complemento} \\ &= x && \text{elemento neutro} \end{aligned}$$

L'idempotenza di $+$ ($x + x = x$) segue per dualità.

1.2 Algebra di commutazione

Definizione. [Algebra di commutazione (*switching algebra*)] È l'algebra di Boole il cui supporto è costituito dai soli valori $\{0, 1\}$:

$$\mathcal{B} = \langle \{0, 1\}, +, \cdot, ', 0, 1 \rangle.$$

Ogni variabile può assumere solo i valori 0 e 1.

È l'algebra usata da Shannon per l'analisi dei circuiti. I tre operatori sono definiti dalle seguenti tabelle di verità.

$+$	0	1
0	0	1
1	1	1

$\cdot$	0	1
0	0	0
1	0	1

x	x'
0	1
1	0

1.2.1 OR esclusivo

L'*OR esclusivo* (XOR) è l'operatore $\oplus$ definito da

$$a \oplus b = a'b + ab',$$

cioè vale 1 quando a e b sono diversi.

$\oplus$	0	1
0	0	1
1	1	0

Dalla definizione, applicando le proprietà dell'algebra di Boole, si ricava:

$$x \oplus 1 = x' \qquad x \oplus 0 = x \tag{1}$$

$$x \oplus x = 0 \qquad x \oplus x' = 1 \tag{2}$$

$$x' \oplus y = x \oplus y' = (x \oplus y)' \tag{3}$$

Esercizi per casa

Esercizio. Usare i diagrammi di Venn per dimostrare le proprietà (associatività, idempotenza, ...) dell'algebra booleana su supporto finito.

Esercizio. Dimostrare che l'algebra di commutazione è un'algebra di Boole, cioè verificare i postulati di Huntington.

Esercizio. Dimostrare, per l'algebra di commutazione, associatività, idempotenza, elemento nullo, unicità del complemento, assorbimento, semplificazione, involuzione, De Morgan e consenso.

2 Funzioni ed espressioni booleane

Lezione 2 – 6 ottobre 2026

2.1 Funzioni booleane

Definizione. [Funzione booleana] Sia $\mathcal{B} = \langle A = \{0,1\}, +, \cdot, ', 0, 1\rangle$ un'algebra di commutazione. Una *funzione booleana* a n variabili è una relazione

$$f\colon A^n \to A$$

che mette in corrispondenza gli elementi del dominio A^n con gli elementi del codominio A.

Dal punto di vista operativo una funzione booleana si definisce con le quattro regole seguenti.

1. $\forall a \in A$, la *funzione costante* $f(x_1, \ldots, x_n) = a$ è una funzione booleana a n variabili.
2. $\forall x_i \in \{x_1, \ldots, x_n\}$, la *funzione proiezione* $f(x_1, \ldots, x_n) = x_i$ è una funzione booleana a n variabili.
3. Se g e h sono funzioni booleane a n variabili, lo sono anche $g + h$, $g \cdot h$ e g', definite da
$$\begin{aligned}(g+h)(x_1,\ldots,x_n) &= g(x_1,\ldots,x_n) + h(x_1,\ldots,x_n)\\ (g\cdot h)(x_1,\ldots,x_n) &= g(x_1,\ldots,x_n)\cdot h(x_1,\ldots,x_n)\\ (g')(x_1,\ldots,x_n) &= \big(g(x_1,\ldots,x_n)\big)'\end{aligned}$$
4. Non esistono altre funzioni booleane oltre a quelle ottenute dall'applicazione iterativa delle regole precedenti.

2.1.1 Tabelle di verità

In un'algebra di commutazione $A = \{0,1\}$, quindi le funzioni booleane hanno la forma

$$f\colon \{0,1\}^n \to \{0,1\}.$$

Il dominio di f ha 2^n elementi e il codominio ne ha 2. La funzione può essere descritta dalla sua *tabella di verità* T_f, che contiene una riga per ciascuna configurazione delle n variabili in ingresso, con il relativo valore di uscita di f.

Esempio. Funzione booleana sulle variabili x, y, z che vale 1 se e solo se *esattamente due* variabili valgono 1.

x	y	z	$f(x,y,z)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Proposizione. Il numero di funzioni booleane (su un'algebra di commutazione) di n variabili è 2^{2^n}.

Infatti, date n variabili si hanno 2^n configurazioni (tante quanti gli elementi di $\{0,1\}^n$) e ogni funzione è definita dall'insieme delle configurazioni per cui assume valore 1. Il numero di funzioni è quindi la cardinalità dell'insieme delle parti di $\{0,1\}^n$, cioè 2^{2^n}.

2.1.2 Esempi

Esempio (Funzioni di una variabile)**.** Alcune di queste hanno particolare importanza dal punto di vista della realizzazione.

x	$f(x)$	$f(x)$	$f(x)$	$f(x)$
0	0	0	1	1
1	0	1	0	1
	costante 0	proiezione di x	negazione di x	costante 1

Esempio (Funzioni di due variabili (I))**.**

x	y	$f(x,y)$					
0	0	0	1	0	0	1	1
0	1	0	1	0	1	1	0
1	0	0	1	1	0	0	1
1	1	0	1	1	1	0	0
		costante 0	costante 1	proiez. x	proiez. y	negaz. x	negaz. y

Esempio (Funzioni di due variabili (II))**.**

x	y	$f(x,y)$					
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1
		AND	OR	NAND	NOR	XOR	EQUIV.

2.1.3 Funzioni non completamente specificate

Talvolta una funzione f viene specificata solo in parte e si dice *non completamente specificata*. Ci sono due motivi principali:

1. la natura del problema garantisce che una certa combinazione delle variabili di ingresso non possa presentarsi;
2. non ha importanza quale valore assuma la funzione per una certa combinazione delle variabili di ingresso.

Nel primo caso si dice che la funzione *non è controllabile* in corrispondenza di quella combinazione di ingressi, nel secondo che *non è osservabile*.

Esempio. Sia f una funzione di due variabili x_1, x_2 tale che

$$\big(f(x_1,x_2)=1 \iff x_1=x_2=0\big) \wedge \big(f(x_1,x_2)=0 \iff x_1 \neq x_2\big).$$

Il valore di $f(1,1)$ non è specificato (nella tabella si indica con $-$).

x_1	x_2	$f(x_1,x_2)$
0	0	1
0	1	0
1	0	0
1	1	$-$

2.2 Espressioni booleane

Definizione. [Espressione booleana] Un'*espressione booleana* definita su un insieme di variabili booleane $X = \{x_1, x_2, \ldots, x_n\}$ è definita dalle regole seguenti:

- gli elementi del supporto sono espressioni;
- le variabili $x_1, x_2, \ldots, x_n$ sono espressioni;
- se h, g sono espressioni booleane, lo sono anche $h + g$, $h \cdot g$ e h';
- non esistono altre espressioni oltre a quelle ottenute applicando iterativamente le regole precedenti.

2.2.1 Notazioni e convenzioni

Fissato l'insieme dei simboli usati per le variabili, si può eliminare il simbolo di prodotto logico $\cdot$ nei termini prodotto. Per le proprietà dell'algebra booleana si possono inoltre omettere alcune parentesi, ricordando che il prodotto logico ha *priorità* sulla somma logica.

Esempio.

- $x + (y \cdot z)$ si scrive $x + yz$ (si elimina il simbolo $\cdot$ nel termine prodotto $y \cdot z$ e si usa la priorità di $\cdot$ rispetto a $+$);
- per la proprietà associativa, $x + (y + z \cdot x)$ si scrive $x + y + zx$ (si eliminano le parentesi per l'associatività e il simbolo $\cdot$ nel termine prodotto $z \cdot x$).

2.2.2 Letterali

Un *letterale* è un'occorrenza di una variabile (in forma naturale o negata) all'interno di un'espressione.

Esempio. L'espressione $(x' + y') \cdot (x + x')$ contiene 4 letterali:

- un'occorrenza della variabile x in forma naturale;
- due occorrenze della variabile x in forma negata (x');
- un'occorrenza della variabile y in forma negata (y').

2.2.3 Valutazione di un'espressione

Sia e un'espressione booleana sulle variabili $x_1, \ldots, x_n$ e sia $(v_1, \ldots, v_n) \in \{0,1\}^n$ una configurazione delle variabili. La *valutazione* di e rispetto a $(v_1, \ldots, v_n)$ è il procedimento con cui si sostituisce a ciascuna variabile il suo valore e, applicando le regole dell'algebra di commutazione, si ottiene un valore $v \in \{0,1\}$ per l'espressione. Ripetendo il procedimento per tutte le configurazioni si ottiene una tabella di verità: *ad ogni espressione corrisponde dunque una funzione.*

Esempio. Sia $e = z(y + x) + xy$. La valutazione per $(1,0,0)$ è

$$e(100) = 0 \cdot (0 + 1) + 1 \cdot 0 = 0.$$

La valutazione completa determina la tabella di verità della funzione booleana su x, y, z:

x	y	z	e
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

2.2.4 Espressioni e funzioni

Ad una singola espressione corrisponde una singola funzione; ad una funzione possono invece corrispondere molte espressioni.

Definizione. [Espressioni equivalenti] Due espressioni e_1, e_2 definite sullo stesso insieme di variabili si dicono *equivalenti*, e si scrive $e_1 \equiv e_2$, se corrispondono alla stessa funzione.

Le trasformazioni eseguite su un'espressione di partenza e_0 usando i postulati e le proprietà dell'algebra booleana portano a espressioni equivalenti.

2.2.5 Espressioni a confronto

Nei contesti in cui si usa un'algebra booleana è fondamentale rappresentare le funzioni con *buone* espressioni. Un possibile criterio di bontà consiste nel minimizzare il numero di letterali: si individua una prima espressione che rappresenti la funzione e si cerca poi un'espressione equivalente migliore (con meno letterali). Espressioni di partenza tipiche da manipolare per minimizzare i letterali sono le espressioni in *forma canonica*. Le forme canoniche di base sono due: la prima espressa come somma di prodotti (SOP), la seconda come prodotto di somme (POS); entrambe si ottengono dal teorema di espansione di Shannon.

2.3 Forme canoniche

2.3.1 Teorema di espansione di Shannon

Teorema (Espansione di Shannon). Sia $f\colon B^n \to B$ una funzione booleana di n variabili. Allora per ogni $(x_1, \ldots, x_n) \in B^n$

$$f(x_1, \ldots, x_n) = x_1' \cdot f(0, x_2, \ldots, x_n) + x_1 \cdot f(1, x_2, \ldots, x_n)$$

e dualmente

$$f(x_1, \ldots, x_n) = \big(x_1' + f(1, x_2, \ldots, x_n)\big) \cdot \big(x_1 + f(0, x_2, \ldots, x_n)\big).$$

I termini $f(0, x_2, \ldots, x_n)$ e $f(1, x_2, \ldots, x_n)$ si chiamano *cofattori* e si indicano spesso con $f_{x_1'}$ e f_{x_1}.

Esempio (Applicazione iterata, forma somma)**.** Per una funzione di due variabili $f(x, y)$:

$$\begin{aligned} f(x, y) &= x' \cdot f(0, y) + x \cdot f(1, y) \\ &= x'\big(y' \cdot f(0, 0) + y \cdot f(0, 1)\big) + x\big(y' \cdot f(1, 0) + y \cdot f(1, 1)\big) \\ &= x'y' \cdot f(00) + x'y \cdot f(01) + xy' \cdot f(10) + xy \cdot f(11). \end{aligned}$$

In generale, l'applicazione iterata del teorema a una funzione di n variabili permette di riscriverla come somma di 2^n termini:

$$\begin{aligned} f(x_1,\dots,x_n) = {} & x_1'\cdots x_{n-1}'x_n' \cdot f(0\dots00) \\ & + x_1'\cdots x_{n-1}'x_n \cdot f(0\dots01) \\ & + \cdots \\ & + x_1\cdots x_{n-1}x_n \cdot f(1\dots11). \end{aligned}$$

Esempio (Applicazione iterata, forma prodotto)**.** Applicando la forma duale a $f(x,y)$:

$$\begin{aligned} f(x,y) &= \big(x' + f(1,y)\big)\big(x + f(0,y)\big) \\ &= \big(x' + [(y' + f(11))(y + f(10))]\big)\big(x + [(y' + f(01))(y + f(00))]\big) \\ &= (x' + y' + f(11))(x' + y + f(10))(x + y' + f(01))(x + y + f(00)). \end{aligned}$$

In generale si ottiene il prodotto di 2^n termini:

$$\begin{aligned} f(x_1,\dots,x_n) = {} & (x_1 + \cdots + x_{n-1} + x_n + f(0\dots00)) \\ & \cdot(x_1 + \cdots + x_{n-1} + x_n' + f(0\dots01)) \\ & \cdot\cdots \\ & \cdot(x_1' + \cdots + x_{n-1}' + x_n' + f(1\dots11)). \end{aligned}$$

2.3.2 Prima forma canonica: mintermini

Definizione. [Mintermine] Un *mintermine* su $x_1,\dots,x_n$ è un termine prodotto $y_1\cdot y_2\cdots y_n$ dove $y_i = x_i$ oppure $y_i = x_i'$, per ogni $i = 1,\dots,n$.
Dato $\bar{v} = (v_1,\dots,v_n) \in \{0,1\}^n$ si costruisce il mintermine $m_{\bar{v}} = y_1 \cdot y_2 \cdots y_n$ con $y_i = x_i$ se $v_i = 1$ e $y_i = x_i'$ se $v_i = 0$ (il mintermine *corrispondente* alla configurazione $\bar{v}$).

Il mintermine $m_{\bar{v}}$ è un'espressione che vale 1 in corrispondenza dell'*unica* configurazione di ingresso $\bar{v}$.

Esempio. Sia $\bar{v} = (101)$ una configurazione per le variabili $\{x,y,z\}$. Il mintermine associato è $m_{\bar{v}} = xy'z$, che assume valore 1 soltanto se $x = 1$, $y = 0$ e $z = 1$, cioè per la sola configurazione (101).

Definizione. [Prima forma canonica (*Sum of Products*, SOP)] Data una funzione booleana $f(x_1,\dots,x_n)$, la sua rappresentazione in *prima forma canonica* è la somma di tutti i mintermini su $x_1,\dots,x_n$ associati alle configurazioni per cui f vale 1.

L'applicazione iterata del teorema di Shannon porta alla prima forma canonica. In pratica, per ricavarla basta guardare le righe della tabella di verità dove f vale 1.

Esempio. Sia f la funzione su x,y,z che vale 1 se e solo se esattamente due variabili valgono 1 (tabella dell'esempio all'inizio della lezione). I mintermini associati alle configurazioni per cui $f = 1$ sono:

- $x'yz$, dalla quarta riga (011);
- $xy'z$, dalla sesta riga (101);
- xyz', dalla settima riga (110).

Quindi la prima forma canonica di f è

$$f = x'yz + xy'z + xyz'.$$

2.3.3 Seconda forma canonica: maxtermini

Definizione. [Maxtermine] Un *maxtermine* su $x_1, \ldots, x_n$ è un termine somma $y_1 + y_2 + \cdots + y_n$ dove $y_i = x_i$ oppure $y_i = x_i'$, per ogni $i = 1, \ldots, n$.
Data la configurazione $\bar{v} = (v_1, \ldots, v_n) \in \{0,1\}^n$ si costruisce il maxtermine $M_{\bar{v}} = y_1 + y_2 + \cdots + y_n$ con $y_i = x_i$ se $v_i = 0$ e $y_i = x_i'$ se $v_i = 1$ (il maxtermine *corrispondente* alla configurazione $\bar{v}$).

Il maxtermine $M_{\bar{v}}$ è un'espressione che vale 0 in corrispondenza dell'*unica* configurazione di ingresso $\bar{v}$.

Esempio. Per $\bar{v} = (101)$ su $\{x, y, z\}$ il maxtermine associato è $M_{\bar{v}} = x' + y + z'$, che assume valore 0 soltanto se $x = 1$, $y = 0$ e $z = 1$.

Definizione. [Seconda forma canonica (*Product of Sums*, POS)] Data una funzione booleana $f(x_1, \ldots, x_n)$, la sua rappresentazione in *seconda forma canonica* è il prodotto di tutti i maxtermini su $x_1, \ldots, x_n$ corrispondenti alle configurazioni per cui f vale 0.

L'applicazione iterata del teorema di Shannon nella sua forma duale porta alla seconda forma canonica. In pratica, per ricavarla basta guardare le righe della tabella di verità dove f vale 0.

Esempio. Per la stessa funzione (esattamente due variabili valgono 1) i maxtermini associati alle configurazioni per cui $f = 0$ sono:

- $x + y + z$, dalla prima riga (000);
- $x + y + z'$, dalla seconda riga (001);
- $x + y' + z$, dalla terza riga (010);
- $x' + y + z$, dalla quinta riga (100);
- $x' + y' + z'$, dall'ultima riga (111).

Quindi la seconda forma canonica di f è

$$f = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)(x' + y' + z').$$

Riferimenti

Capitolo 2 del libro di testo: C. Bolchini et al., *Reti Logiche*, Maggioli Editore. Lezione 2: sezione 2.3; lezione 3: sezioni 2.4 e 2.5.

Esercizi per casa

Esercizio. Definire la tabella di verità della funzione booleana $f\colon B^3 \to B$ che assume valore 1 soltanto se almeno due bit in ingresso hanno valore 0.

Esercizio. Rappresentare f mediante un'espressione booleana in prima forma canonica.

Esercizio. Rappresentare f' mediante un'espressione booleana in seconda forma canonica.

3 Porte logiche e circuiti logici

Lezione 3

3.1 Porte logiche

Definizione. [Porta logica] Una *porta logica* è un componente elettronico di base che opera su uno o più segnali di ingresso e genera un unico segnale in uscita. È la realizzazione fisica delle funzioni elementari introdotte dall'algebra (AND, OR, NOT).

Oltre alle porte AND, OR e NOT, si usano le porte NAND, NOR, XOR e XNOR.

x_0	x_1	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

La porta NOT ha un solo ingresso e restituisce x'.

3.1.1 NAND e NOR: operatori funzionalmente completi

La porta NAND realizza $(x_0 \cdot x_1)'$ e la porta NOR realizza $(x_0 + x_1)'$.

Proposizione. NAND e NOR sono *funzionalmente completi*: qualsiasi funzione logica si può realizzare usando esclusivamente porte NAND, oppure esclusivamente porte NOR, invece di AND, OR e NOT.

Esempio (NOT con una NAND)**.** Collegando i due ingressi di una NAND alla stessa variabile x si ottiene, per idempotenza,

$$(x \cdot x)' = (x)' = x'.$$

Quindi la porta NOT si realizza con una NAND.

3.1.2 XOR e XNOR

Le porte XOR e XNOR corrispondono agli operatori booleani XOR e XNOR (OR e NOR *esclusivi*): realizzano rispettivamente la funzione di *disuguaglianza* e di *uguaglianza* tra due bit. Lo XOR vale 1 se e solo se i due ingressi sono diversi, lo XNOR vale 1 se e solo se sono uguali.

3.1.3 Porte a n ingressi

Accanto alle porte elementari a due ingressi, in genere si assume di avere a disposizione le stesse porte con un numero arbitrario di ingressi. In una AND a tre ingressi x_0, x_1, x_2 l'uscita y vale 1 se e solo se tutte le variabili in ingresso valgono 1. In una OR a quattro ingressi $x_0, \ldots, x_3$ l'uscita y vale 1 se almeno una variabile in ingresso vale 1.

3.2 Circuiti logici

Le porte elementari si collegano per realizzare funzioni più complesse. A partire da un'espressione logica e (ricavata, per esempio, dalla tabella di verità di una funzione booleana) si costruisce il circuito che realizza la funzione descritta da e.

Esempio. Il circuito per $x + y'z$ usa una porta NOT su y, una AND tra y' e z (uscita $y'z$) e una OR tra x e $y'z$.

3.2.1 Ritardi e cammino critico

Ogni porta logica ha un *ritardo*: il tempo tra l'istante in cui si applicano i segnali in ingresso e l'istante in cui il segnale di uscita si stabilizza a un valore costante. Il ritardo dipende dalla realtà fisica dei componenti (i transistor).

> **Definizione.** [Cammino critico] Il *cammino critico* (*critical path*) è il cammino dagli ingressi alle uscite che attraversa il maggior numero di porte in un circuito, trascurando i negatori, perché possono essere accorpati alla porta precedente o successiva.

Esempio. In una rete divisa in tre livelli di porte, i cammini critici attraversano tre porte.

3.2.2 Circuiti da forme canoniche

Le funzioni espresse in prima o in seconda forma canonica corrispondono sempre a circuiti logici *a due livelli*, che usano porte a n ingressi.

- In *prima forma canonica* il primo livello contiene le porte AND che calcolano i mintermini e il secondo livello un'unica porta OR che ne fa la somma.
- In *seconda forma canonica* il primo livello contiene le porte OR che calcolano i maxtermini e il secondo livello un'unica porta AND che ne fa il prodotto.

Esempio. Sia $f(x, y, z)$ la funzione che vale 1 solo per 011, 101, 110:

x	y	z	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

La prima forma canonica è $f = x'yz + xy'z + xyz'$. Il circuito ha tre NOT per le variabili negate, tre AND a tre ingressi (una per mintermine) e una OR a tre ingressi.

3.3 Circuiti speciali: i sommatori

Con le porte logiche si costruiscono circuiti che eseguono operazioni aritmetiche. Vediamo tre sommatori.

3.3.1 Half adder

Un *half adder* (HA) ha come ingressi due bit a, b e come uscite due bit: s (la somma) e c (il riporto, *carry*).

a	b	s	c
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

La colonna s è uno XOR, la colonna c è un AND:

$$s = a \oplus b, \qquad c = a \cdot b.$$

Il circuito è quindi formato da una porta XOR e una porta AND con gli stessi ingressi.

3.3.2 Full adder

Un *full adder* (FA) ha tre ingressi, a, b e c_{in} (bit e riporto in ingresso), e due uscite, s e c_{out} (somma e riporto in uscita).

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Poiché la somma è associativa, $a + b + c_{in} = (a + b) + c_{in}$: si possono usare due half adder, il primo per $a + b$ e il secondo per sommare c_{in} al risultato, e combinare i due riporti con una OR per ottenere c_{out}.

3.3.3 Sommatore a n bit (ripple-carry)

Un *sommatore a n bit* ha come ingressi $2n$ bit, $a_{n-1} \dots a_0$ e $b_{n-1} \dots b_0$, e come uscite n bit $s_{n-1} \dots s_0$ più il riporto finale c, tali che la sequenza in uscita rappresenti in binario la somma dei due numeri in ingresso. Si realizza collegando n full adder in cascata: il riporto in uscita di ciascuno entra come riporto in ingresso del successivo (da cui il nome *ripple-carry*).

Esempio. Per sommare due numeri a 3 bit $X = x_2x_1x_0$ e $Y = y_2y_1y_0$ servono 3 full adder in cascata. Ad esempio $100 + 110 = 1010$.

Riferimenti. Libro di testo (C. Bolchini et al., *Reti Logiche*, Maggioli): cap. 2, sottosezioni 2.6 (porte logiche) e 2.7 (circuiti logici); cap. 5, sottosezione 5.2.1 (sommatori ripple-carry). Esercizi 2.24, 2.25, 2.26, 2.27.

4 Progettazione di reti logiche combinatorie

Lezione 4 Una rete logica *combinatoria* ha un comportamento che dipende solo dagli ingressi, perché è priva di memoria. Vediamo come si progetta e si ottimizza.

4.1 Le fasi della progettazione

Progettare un circuito combinatorio comporta quattro passi successivi.

1. **Comprensione e analisi della specifica semi-formale.** La descrizione iniziale di un problema spesso non è completa e va riscritta per eliminare ogni ambiguità. Casi tipici: gestione delle condizioni limite o dei casi di errore, analisi dell'ambiente in cui la rete opera, scelta della codifica delle informazioni. Alla fine si ha ancora una specifica semi-formale, ma completa.

2. **Formalizzazione della specifica.** Si traduce la specifica in un linguaggio formale, più adatto alla manipolazione: le *tabelle di verità*, che specificano la relazione richiesta tra ingressi e uscite.
3. **Sintesi.** Si passa dalle tabelle di verità a espressioni algebriche booleane. In questa fase si trascura ogni considerazione sulla qualità.
4. **Ottimizzazione.** Fissato un criterio di qualità, si manipola l'espressione ottenuta nella sintesi per avere un'espressione funzionalmente equivalente ma migliore.

Le caratteristiche che misurano la qualità di una rete sono generalmente l'area della realizzazione fisica, il tempo necessario a produrre un risultato, la potenza e l'energia assorbita (*metriche d'area*, *metriche di tempo*, ...). Alcune metriche d'area comuni sono il numero di porte logiche generiche, il numero di porte a due ingressi, il numero di implicanti/implicati e il numero di letterali.

4.2 Esempio: riconoscere pari e dispari

Si progetti una rete combinatoria in grado di discriminare i numeri pari dai numeri dispari.

Fase 1: specifica

- Ci interessano i numeri interi e dobbiamo fissare l'intervallo, in base all'applicazione. Scegliamo per semplicità i giorni della settimana, cioè l'intervallo $[0, 6]$.
- Servono $\lceil \log_2 7 \rceil = 3$ bit per rappresentarlo.
- L'uscita è binaria (pari o dispari), quindi basta un bit. Fissiamo la codifica 1 = pari e 0 = dispari.
- Con l'intervallo $[0, 6]$ l'ingresso 111 (il numero 7) non è valido. Introduciamo un secondo bit di uscita per segnalare l'errore.

La specifica semi-formale completa è: *una rete combinatoria con un ingresso a tre bit* $X = \{x_0, x_1, x_2\}$ *che rappresenta un intero in* $[0, 6]$ *in codifica binaria naturale e due uscite di un bit ciascuna:* z *indica se il valore è pari* ($z = 1$) *o dispari* ($z = 0$)*, e indica un errore* ($e = 1$) *o il funzionamento corretto* ($e = 0$)*.*

Fase 2: formalizzazione

La rete realizza le funzioni $z(x_0, x_1, x_2)$ ed $e(x_0, x_1, x_2)$, con x_0 bit più significativo:

x_0	x_1	x_2	z	e
0	0	0	1	0
0	0	1	0	0
0	1	0	1	0
0	1	1	0	0
1	0	0	1	0
1	0	1	0	0
1	1	0	1	0
1	1	1	0	1

Fase 3: sintesi

Dalle tabelle si ricavano le espressioni in prima forma canonica:

$$z = x_0' x_1' x_2' + x_0' x_1 x_2' + x_0 x_1' x_2' + x_0 x_1 x_2',$$
$$e = x_0 x_1 x_2.$$

Da qui si potrebbe già derivare un circuito, ma le proprietà dell'algebra booleana permettono di ottimizzare (in numero di letterali).

Fase 4: ottimizzazione

Raccogliendo x_2' e usando $y + y' = 1$ due volte:

$$z = x_2' \left(x_0' x_1' + x_0' x_1 + x_0 x_1' + x_0 x_1\right) = x_2' \left(x_0'(x_1' + x_1) + x_0(x_1' + x_1)\right) = x_2'.$$

Quindi $z = x_2'$: un numero è pari se e solo se l'ultimo bit è 0. Le tecniche sistematiche di ottimizzazione sono oggetto delle lezioni successive (mappe di Karnaugh).

4.3 Secondo esempio: display a sette segmenti

Un display a 7 segmenti è costituito da 7 segmenti luminosi che si accendono o spengono a formare il simbolo di una cifra; per controllarlo servono 7 segnali binari. Si progetti una rete combinatoria che gestisca un display a sette segmenti di una cifra decimale nell'intervallo $[0, 7]$ in codifica binaria naturale: la rete ha 3 ingressi e 7 uscite, una per segmento.

Riferimenti. Libro di testo (C. Bolchini et al., *Reti Logiche*, Maggioli): cap. 4, sottosezioni 4.1, 4.2, 4.3. Esercizi per casa: 4.1, 4.2, 4.5, 4.6, 4.7, 4.9.

5 Ottimizzazione: mappe di Karnaugh

Lezione 5

5.1 Che cos'è una mappa di Karnaugh

Le mappe di Karnaugh sono un metodo *grafico* per semplificare le espressioni booleane. Furono inventate nel 1953 da Maurice Karnaugh, ingegnere delle telecomunicazioni dei Bell Labs, e sono molto efficaci per ottimizzare reti combinatorie con poche variabili (fino a quattro).

> **Definizione.** [Mappa di Karnaugh] Una *mappa di Karnaugh* è una rappresentazione formale, alternativa alla tabella di verità, di una funzione booleana: una griglia con una cella per ogni configurazione delle variabili, in cui si scrive il valore della funzione.

Esempio. Sia $f(x, y, z)$ con $f(001) = f(011) = 1$ e $f = 0$ altrove. Le righe della mappa indicano x e le colonne indicano yz:

	00	01	11	10
0	0	1	1	0
1	0	0	0	0

Le due celle con 1 sono quelle di $f(001)$ e $f(011)$.

5.2 Codice di Gray

Nella mappa le configurazioni delle variabili non seguono l'ordine binario naturale $(00, 01, 10, 11)$ ma un *codice di Gray* (o codice a specchio): $00, 01, 11, 10$. Passando da una colonna alla successiva cambia un solo bit. Ciò è fondamentale perché mette in celle *adiacenti* (con un lato in comune) i mintermini che danno luogo a semplificazioni.

Per esempio, nella mappa precedente

$$x'y'z + x'yz = x'z\,(y' + y) = x'z.$$

5.3 Perché funzionano

Due mintermini sono *logicamente adiacenti* se sono uguali a meno di una variabile, che compare in forma naturale in uno e negata nell'altro (per esempio $x_1x_2'x_3$ e $x_1x_2x_3$). Se $p_1 + p_2$ è la somma di due mintermini logicamente adiacenti, allora $p_1 + p_2$ si può semplificare:

$$x_1x_2'x_3 + x_1x_2x_3 = x_1x_3\,(x_2' + x_2) = x_1x_3.$$

In una mappa di Karnaugh, a celle geometricamente adiacenti corrispondono mintermini logicamente adiacenti.

5.4 Come si usa

1. Si raggruppano gli 1 che compaiono in celle adiacenti: ogni gruppo è un *k-cubo*.
2. Da ogni k-cubo si ricava un *implicante*, cioè un termine prodotto p tale che, se p vale 1, allora f vale 1. L'implicante è il prodotto delle variabili che compaiono sempre in forma naturale (oppure sempre in forma negata) in tutte le celle del k-cubo.
3. Coperti tutti gli 1, si sommano gli implicanti: si ottiene un'espressione ottimizzata, in forma di somma di prodotti, corrispondente a f.

Regole per generare i k-cubi

1. La mappa si immagina *arrotolata* su se stessa: l'ultima colonna è adiacente alla prima e l'ultima riga è adiacente alla prima.
2. I lati dei rettangoli in cui si raccolgono gli 1 devono essere potenze di 2.
3. Ogni k-cubo deve essere il più grande possibile.
4. Ogni 1 deve stare in almeno un k-cubo.
5. Un 1 può stare in più k-cubi, se serve per coprire tutti gli 1.

5.5 Esempi

Esempio (Tre variabili)**.** Mappa con righe x e colonne yz e $f = y'z' + yz$:

	00	01	11	10
0	1	0	1	0
1	1	0	1	0

I due k-cubi sono la colonna 00 (implicante $y'z'$) e la colonna 11 (implicante yz).

Esempio (Tre variabili, con la colonna 10)**.**

	00	01	11	10
0	0	1	1	1
1	1	1	0	1

Si hanno tre k-cubi: le celle $01, 11$ della riga 0 (implicante $x'z$), le celle $00, 01$ della riga 1 (implicante xy') e la colonna 10 (implicante yz'). Quindi

$$f(x, y, z) = x'z + xy' + yz'.$$

Esempio (Quattro variabili)**.** Righe xy, colonne zw:

	00	01	11	10
00	0	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	0

I k-cubi sono: le righe 11 e 10 nelle colonne $00, 01$ (implicante xz'); le celle 01 e 11 della colonna 01 (implicante $yz'w$); le celle 00 e 01 della colonna 10 (implicante $x'zw'$). Quindi

$$f(x, y, z, w) = xz' + yz'w + x'zw'.$$

Esempio (Quattro variabili, con celle ai bordi)**.**

	00	01	11	10
00	1	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	0

Rispetto all'esempio precedente la cella $(00, 00)$ vale 1. Va raccolta con la cella $(10, 00)$, perché la prima e l'ultima riga sono adiacenti: si ottiene l'implicante $y'z'w'$. Quindi

$$f(x, y, z, w) = xz' + yz'w + x'zw' + y'z'w'.$$

Esempio (Quattro variabili, cubo grande)**.**

	00	01	11	10
00	1	0	1	1
01	0	0	1	1
11	0	0	1	1
10	1	0	1	1

Le colonne 11 e 10 formano un k-cubo 4×2 (implicante z). Le quattro celle d'angolo (righe $00, 10$ e colonne $00, 10$) formano un secondo k-cubo, perché la mappa è arrotolata sia in orizzontale sia in verticale (implicante $y'w'$). Quindi

$$f(x, y, z, w) = z + y'w'.$$

6 Mappe di Karnaugh: il punto di vista del libro

Lezione 6

6.1 Mappe di Karnaugh: il punto di vista del libro

Questa lezione riprende le mappe di Karnaugh (lezione 5) seguendo il libro di testo di Harris & Harris, *Digital Design and Computer Architecture*, sezione 2.7.

Dopo aver provato a minimizzare equazioni booleane con l'algebra di Boole, ci si accorge che, se non si sta attenti, si può finire con un'equazione completamente diversa invece che semplificata. Le mappe di Karnaugh (*K-map*) sono un metodo grafico per semplificare le equazioni booleane; furono inventate nel 1953 da Maurice Karnaugh, ingegnere delle telecomunicazioni ai Bell Labs. Funzionano bene per problemi con fino a quattro variabili e, soprattutto, aiutano a capire come manipolare le equazioni booleane.

Osservazione. Maurice Karnaugh (1924–) si è laureato in fisica al City College di New York nel 1948 e ha conseguito il dottorato in fisica a Yale nel 1952. Ha lavorato ai Bell Labs e all'IBM dal 1952 al 1993 e come professore di informatica al Polytechnic University di New York dal 1980 al 1999.

6.2 Perché una mappa

Minimizzare una funzione logica significa combinare dei termini. Due termini che contengono un implicante P e la forma vera e quella complementare di una stessa variabile A si combinano eliminando A:

$$PA + PA' = P.$$

Le mappe di Karnaugh rendono evidenti questi termini combinabili, mettendoli uno accanto all'altro in una griglia.

6.2.1 Mappa di una funzione a tre ingressi

Consideriamo la funzione Y di tre ingressi A, B, C con $Y = 1$ solo per $ABC = 000$ e $ABC = 001$ (e 0 altrove). La riga superiore della mappa riporta i quattro possibili valori degli ingressi A e B; la colonna di sinistra riporta i due possibili valori dell'ingresso C. Ogni quadrato della mappa corrisponde a una riga della tabella di verità e contiene il valore dell'uscita Y per quella riga.

	AB			
C	00	01	11	10
0	1	0	0	0
1	1	0	0	0

Come ogni riga di una tabella di verità, ogni quadrato di una K-map rappresenta un singolo *mintermine*: la cella in alto a sinistra è $A'B'C'$, quella sotto è $A'B'C$, e così via per tutti gli otto mintermini.

6.3 Codice di Gray e adiacenza

Ogni quadrato (mintermine) differisce da un quadrato adiacente per una sola variabile. Quindi i quadrati adiacenti condividono tutti i letterali tranne uno, che compare in forma vera in un quadrato e in forma complementare nell'altro. Per esempio i quadrati che rappresentano i mintermini $A'BC'$ e $A'BC$ sono adiacenti e differiscono solo per la variabile C.

Le combinazioni di A e B nella riga superiore sono in un ordine particolare: $00, 01, 11, 10$. Questo ordine si chiama *codice di Gray*. Si distingue dal normale ordine binario $(00, 01, 10, 11)$ perché voci adiacenti differiscono per una sola variabile: per esempio passando da 01 a 11 cambia solo A (da 0 a 1), mentre passando da 01 a 10 cambierebbe sia A (da 1 a 0) sia B (da 0 a 1). Scrivere le combinazioni in ordine binario non darebbe la proprietà desiderata, cioè quadrati adiacenti che differiscono per una sola variabile.

Osservazione. I codici di Gray valgono per un numero qualunque di bit. Per esempio una sequenza di Gray a 3 bit è

$$000,\ 001,\ 011,\ 010,\ 110,\ 111,\ 101,\ 100.$$

Furono brevettati nel 1953 (brevetto USA 2 632 058) da Frank Gray, ricercatore ai Bell Labs; sono utili soprattutto negli encoder meccanici, perché un leggero disallineamento provoca un errore in un solo bit. Un problema collegato fu proposto da Lewis Carroll nel 1879: collegare due parole della stessa lunghezza con parole intermedie che differiscono

dalla precedente per una sola lettera, per esempio da SHIP a DOCK: SHIP, SLIP, SLOP, SLOT, SOOT, LOOT, LOOK, LOCK, DOCK.

6.3.1 La mappa si "arrotola"

La K-map si "avvolge" su se stessa: i quadrati all'estrema destra sono di fatto adiacenti a quelli all'estrema sinistra, perché differiscono per una sola variabile, A. In altre parole si può arrotolare la mappa in un cilindro, poi unire le estremità del cilindro per ottenere un toro (una ciambella) e avere comunque la garanzia che quadrati adiacenti differiscano per una sola variabile.

6.4 Minimizzazione con le K-map

6.4.1 Ragionare per cerchi

Nella K-map della funzione precedente nell'equazione compaiono solo due mintermini, $A'B'C'$ e $A'B'C$, indicati dagli 1 nella colonna di sinistra. Leggere i mintermini da una K-map è esattamente equivalente a leggere le equazioni in somma di prodotti direttamente dalla tabella di verità. Come prima si può usare l'algebra di Boole:

$$Y = A'B'C' + A'B'C = A'B'(C' + C) = A'B'.$$

Le K-map permettono di fare questa semplificazione graficamente, cerchiando gli 1 in quadrati adiacenti. Per ogni cerchio si scrive l'*implicante* corrispondente. Un implicante è il prodotto di uno o più letterali; le variabili le cui forme vera e complementare sono entrambe dentro il cerchio vengono escluse dall'implicante. Nel nostro caso la variabile C ha dentro il cerchio sia la forma vera (1) sia la forma complementare (0), quindi non viene inclusa nell'implicante: Y è vera quando $A = B = 0$, indipendentemente da C. L'implicante è $A'B'$, lo stesso risultato ottenuto con l'algebra di Boole.

6.4.2 Regole

La K-map dà un modo visivo semplice per minimizzare la logica: basta cerchiare tutti i blocchi rettangolari di 1 della mappa, usando il minor numero possibile di cerchi. Ogni cerchio deve essere il più grande possibile. Poi si leggono gli implicanti cerchiati.

Più formalmente, un'equazione booleana è minimizzata quando è scritta come somma del minor numero di *implicanti primi*. Ogni cerchio sulla K-map rappresenta un implicante; i cerchi più grandi possibili sono gli implicanti primi.

Definizione. Un *implicante primo* è un implicante che non può essere ulteriormente ingrandito, cioè non può essere combinato con altri mintermini per eliminare una variabile.

Per esempio, nella mappa precedente $A'B'C'$ e $A'B'C$ sono implicanti, ma non primi. Solo $A'B'$ è un implicante primo.

Le regole per trovare un'equazione minimizzata da una K-map sono:

- usare il minor numero di cerchi necessario a coprire tutti gli 1;
- tutti i quadrati dentro ogni cerchio devono contenere 1;
- ogni cerchio deve coprire un blocco rettangolare che ha, in ogni direzione, un numero di quadrati che è una potenza di 2 (1, 2 o 4);
- ogni cerchio deve essere il più grande possibile;
- un cerchio può passare oltre i bordi della K-map (avvolgendosi);
- un 1 in una K-map può essere cerchiato più volte, se questo consente di usare meno cerchi.

Esempio (Minimizzazione di una funzione di tre variabili)**.** Sia $Y = F(A, B, C)$ con la K-map seguente:

	AB			
C	00	01	11	10
0	1	0	1	1
1	1	0	0	1

Si cerchiano gli 1 con il minor numero di cerchi possibile. Ogni cerchio rappresenta un implicante primo, e le dimensioni di ciascun cerchio sono potenze di due (2×1 e 2×2). Si forma l'implicante primo di ciascun cerchio scrivendo le variabili che compaiono nel cerchio solo in forma vera o solo in forma complementare.

- Il cerchio 2×1 comprende i quadrati $AB = 11$ e $AB = 10$ nella riga $C = 0$. In esso compaiono sia B sia B', quindi B non fa parte dell'implicante; compaiono invece solo A in forma vera e C in forma complementare: l'implicante primo è AC'.
- Il cerchio 2×2 copre tutti i quadrati in cui $B = 0$ (le colonne 00 e 10, che sono adiacenti perché la mappa si avvolge ai bordi): l'implicante primo è B'.

Perciò

$$Y = AC' + B'.$$

Si noti che il quadrato in alto a destra ($AB = 10$, $C = 0$) è coperto due volte, per rendere gli implicanti primi i più grandi possibili: come nelle tecniche con l'algebra di Boole, equivale a condividere un mintermine per ridurre la dimensione di un implicante. Si noti anche che il cerchio da quattro quadrati si avvolge attorno ai lati della K-map.

Osservazione. Il libro prosegue con l'esempio 2.10, un decodificatore per display a sette segmenti: un circuito con ingresso a 4 bit $D_{3:0}$ e sette uscite $S_a, \ldots, S_g$ che pilotano i LED per mostrare una cifra da 0 a 9 (gli ingressi illegali 10–15 producono un display spento). Se ne cerca la tabella di verità e, con le K-map, le equazioni delle uscite S_a e S_b. Le pagine fornite con questa lezione terminano all'inizio dell'esempio: vi si ricorda in particolare di etichettare righe e colonne in ordine di Gray anche quando si riportano i valori dell'uscita, e che l'insieme minimo di implicanti primi non è unico.

6.5 Riferimenti

Harris & Harris, *Digital Design and Computer Architecture* (in italiano *Sistemi digitali e architettura dei calcolatori*), sezione 2.7, disponibile su UniStudium (pagine 71–75 fornite). Per le slide della lezione si veda la lezione 5.

7 Reti sequenziali e macchine a stati finiti

Lezione 7

7.1 Reti logiche sequenziali

7.1.1 Circuiti combinatori e circuiti sequenziali

Nelle reti logiche combinatorie viste finora le uscite (il valore delle funzioni calcolate) dipendono *esclusivamente* dalle configurazioni di bit in ingresso.

Nelle reti logiche *sequenziali* invece la configurazione dei bit in ingresso non determina univocamente l'uscita: è necessario portarsi dietro un *riassunto della storia del sistema* per determinare l'uscita.

Esempio (Riconoscitore del codice di Gray)**.** Un circuito sequenziale con due variabili binarie in ingresso (x_0, x_1) e una variabile in uscita z, tale che $z = 1$ se e solo se viene riconosciuto il codice di Gray, cioè si riconosce una sequenza di configurazioni in ingresso nell'ordine

$$(0,0) \to (0,1) \to (1,1) \to (1,0).$$

Esempio (Il semaforo)**.** Si consideri il circuito controllore di un semaforo.

- Il semaforo si può trovare in tre stati: rosso, giallo oppure verde.
- La lampada attualmente accesa dipende dalla temporizzazione associata a rosso, arancione e verde, e dallo stato precedente del semaforo.

7.1.2 Parole chiave

- **Stato**: in pratica, lo stato codifica le informazioni relative alla storia del sistema utili a calcolare l'uscita del sistema.
- **Determinismo**: ci interessano sistemi deterministici: fissati l'ingresso e lo stato, l'uscita e l'evoluzione dello stato sono fissate.
- **Modello del tempo**: in una generica macchina a stati finiti (modello di un sistema fisico) l'evoluzione dello stato è dettata da un *evento*. Se gli eventi sono prodotti da una sorgente indipendente di sincronizzazione (il segnale di *clock*) si parla di macchine a stati *sincrone*, altrimenti di macchine a stati *asincrone*.

7.2 Macchine a stati finiti (FSM)

Le macchine a stati finiti sono un modello matematico per la descrizione di "macchine" dotate di memoria che eseguono algoritmi. Nel nostro contesto permettono di modellare ad alto livello i circuiti sequenziali (per esempio il controllore di un semaforo).

- L'evoluzione del circuito è rappresentata come una sequenza di stati.
- Si è in uno *stato presente* e si evolverà in uno *stato futuro*.
- La realizzazione fisica consiste in una rete (combinatoria) per il calcolo delle uscite e dello stato futuro, a partire da ingressi e stato presente, e in un *registro di attualizzazione* dello stato.

Definizione. [Macchina a stati finiti: macchine di Mealy e di Moore] Una macchina a stati finiti (sincrona) M è una tupla

$$M = \langle S,\, X,\, Z,\, \lambda,\, \delta,\, s_0 \rangle$$

dove:

- $X = \{x_1, \ldots, x_n\}$ è l'*alfabeto di ingresso*;
- $Z = \{z_1, \ldots, z_m\}$ è l'*alfabeto di uscita*;
- $S = \{s_0, \ldots, s_p\}$ è un insieme finito di *stati* e s_0 è lo *stato iniziale*;
- δ e λ sono due funzioni, rispettivamente di *stato prossimo* e di *uscita*:
  - $z_k = \lambda(x_k, s_k)$ e $s_{k+1} = \delta(x_k, s_k)$ per le macchine di **Mealy**;
  - $z_k = \lambda(s_k)$ e $s_{k+1} = \delta(x_k, s_k)$ per le macchine di **Moore**;

dove $x_k \in X$, $z_k \in Z$ e $s_k \in S$ rappresentano rispettivamente il simbolo di ingresso, il simbolo di uscita e lo stato all'istante k.

Dunque in una macchina di Moore l'uscita dipende solo dallo stato; in una macchina di Mealy dipende dallo stato *e* dall'ingresso.

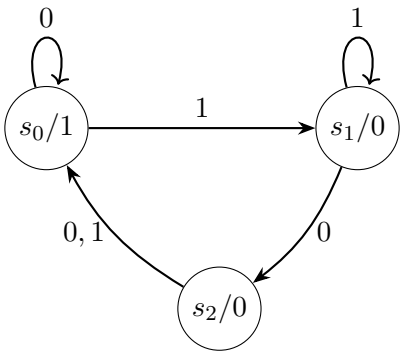
7.2.1 Rappresentazione con grafi di transizione dello stato

I grafi di transizione dello stato sono una rappresentazione grafica molto comoda nelle fasi iniziali di progetto, in cui si passa da una descrizione formale della macchina sequenziale alla sua realizzazione fisica. Si tratta di un grafo orientato in cui ogni stato è rappresentato da un *nodo* e ogni *arco* corrisponde a una transizione dello stato. In particolare ogni transizione (arco) indica il prossimo stato in corrispondenza di un determinato ingresso.

- In una FSM di **Moore** i simboli di uscita sono associati agli *stati* (si scrivono dentro il nodo).
- In una FSM di **Mealy** i simboli di uscita sono associati alle *transizioni* (si scrivono sull'arco, nella forma ingresso/uscita).

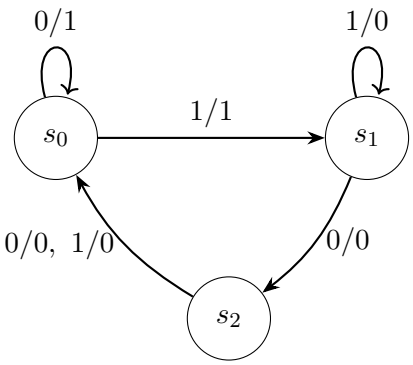
Esempio (Tabella e grafo di una macchina di Moore)**.** Alfabeto di ingresso $\{0, 1\}$, tre stati s_0, s_1, s_2 (iniziale s_0), uscita $U \in \{0, 1\}$.

stato	ingresso 0	ingresso 1	uscita U
s_0	s_0	s_1	1
s_1	s_2	s_1	0
s_2	s_0	s_0	0



Esempio (Tabella e grafo di una macchina di Mealy)**.** Stessi stati e alfabeto; ogni cella della tabella riporta *stato prossimo/uscita*.

stato	ingresso 0	ingresso 1
s_0	$s_0/1$	$s_1/1$
s_1	$s_2/0$	$s_1/0$
s_2	$s_0/0$	$s_0/0$



7.3 Moore e Mealy a confronto

Una macchina di Moore

$$M_{\text{Moore}} = \langle S_{\text{Moore}},\, X,\, Z,\, \lambda_{\text{Moore}},\, \delta_{\text{Moore}},\, s_0 \rangle$$

e una macchina di Mealy

$$M_{\text{Mealy}} = \langle S_{\text{Mealy}},\, X,\, Z,\, \lambda_{\text{Mealy}},\, \delta_{\text{Mealy}},\, s_0 \rangle$$

sono *equivalenti*, dal punto di vista ingresso-uscita, se e solo se, ignorando l'uscita $z_0 = \lambda_{\text{Moore}}(s_0)$ di M_{Moore}, ogni possibile sequenza di simboli in ingresso genera una coppia di sequenze di simboli di uscita, una per M_{Moore} e una per M_{Mealy}, identiche.

7.3.1 Da Moore a Mealy

È sufficiente associare l'uscita appartenente a uno stato a *tutte le transizioni che partono da tale stato.*



7.3.2 Da Mealy a Moore

- Ogni stato della macchina di Mealy va *replicato* tante volte quante sono le transizioni con uscite differenti verso tale stato.
- I nuovi stati avranno il valore di uscita uguale a quello della transizione da cui sono stati originati.

Per esempio, se a uno stato s della macchina di Mealy arrivano transizioni con uscita 0 e transizioni con uscita 1, nella macchina di Moore s diventa una coppia di stati $s/0$ e $s/1$: le transizioni con uscita 0 arrivano in $s/0$, quelle con uscita 1 arrivano in $s/1$, e da ciascuno dei due stati replicati partono le stesse transizioni che partivano da s.

7.4 Esempi

7.4.1 Riconoscere la parola "via"

Esercizio. Si realizzi una macchina a stati finiti che riceve in ingresso una sequenza di caratteri e riconosce la parola "via". L'uscita prodotta è 0 se non si sta riconoscendo la parola, 1 mentre la si sta riconoscendo e 2 una volta che sia stata riconosciuta.

Osservazione. Nelle slide l'esercizio è solo enunciato. Una soluzione possibile (aggiunta qui, non presente nelle slide) è una macchina di Moore con quattro stati: s_0 (nessun carattere riconosciuto, uscita 0), s_1 (visto "v", uscita 1), s_2 (visto "vi", uscita 1), s_3 (visto "via", uscita 2). Da s_0: con "v" si va in s_1, con qualunque altro carattere si resta in s_0. Da s_1: con "i" si va in s_2, con "v" si resta in s_1, altrimenti si torna in s_0. Da s_2: con "a" si va in s_3, con "v" si torna in s_1, altrimenti in s_0. Da s_3: con "v" si va in s_1, altrimenti in s_0.

7.4.2 Il semaforo all'incrocio

Si consideri un incrocio tra due strade, nord-sud (NS) ed est-ovest (EO), controllate da un semaforo.

- Il semaforo può commutare ogni 30 secondi: per semplicità si considerano solamente rosso e verde.
- È presente un sensore in grado di "leggere", per ogni direttrice, se esiste almeno un'auto in attesa oppure un'auto che si accinga ad attraversare (condizioni trattate allo stesso modo).
- Il semaforo deve cambiare colore, da rosso a verde, quando esiste un'auto in attesa sulla sua direttrice.
- Se ci sono auto in attesa su entrambe le direttrici il semaforo deve cambiare colore (al termine del tempo di commutazione).

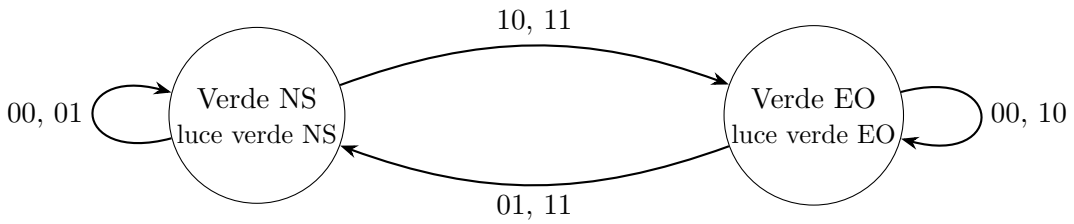
Ingressi. Auto NS presente/assente: $i_{NS} = 1/0$. Auto EO presente/assente: $i_{EO} = 1/0$.

Stati. Semaforo NS verde (quindi semaforo EO rosso); semaforo EO verde (quindi semaforo NS rosso).

Uscite. Luce NS verde (quindi luce EO rossa); luce EO verde (quindi luce NS rossa). Nel nostro caso le uscite coincidono con gli stati.

Tabella di transizione (ingresso (i_{EO}, i_{NS}); uscita = luce verde dello stato):

stato S	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$	uscita
Verde NS	Verde NS	Verde NS	Verde EO	Verde EO	luce verde NS
Verde EO	Verde EO	Verde NS	Verde EO	Verde NS	luce verde EO



Le etichette sugli archi sono le coppie (i_{EO}, i_{NS}). La macchina è di Moore: l'uscita dipende solo dallo stato.

7.5 Riferimenti ed esercizi per casa

- Capitolo 6 del libro di testo: C. Bolchini et al., *Reti Logiche*, Maggioli Editore. Tutto il capitolo tranne le sottosezioni 6.3.3, 6.3.4, 6.3.5.
- Esercizio 6.1 del libro di testo (le soluzioni sono sul libro di testo).

8 Bistabili, latch, flip-flop e sintesi di circuiti sequenziali

Lezione 8

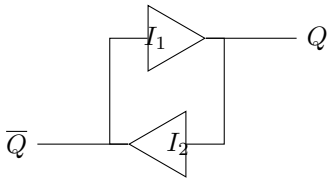
8.1 Elementi bistabili

Il blocco costitutivo fondamentale di un registro di memoria è un elemento *bistabile*: un elemento circuitale con due stati stabili, in grado dunque di memorizzare un bit di informazione. Un elemento con N stati stabili conserva $\log_2 N$ bit.

Definizione. [Stato stabile] Dato un circuito (sequenziale), si dice *stato stabile* del circuito un'attribuzione di valori di verità a tutti i fili del circuito in accordo con il comportamento delle porte logiche.

8.1.1 Due negatori collegati a croce

Il bistabile più semplice è formato da una coppia di negatori I_1, I_2 collegati a croce: l'ingresso di I_1 è l'uscita di I_2 e viceversa.



L'analisi di un elemento sequenziale differisce da quella di un circuito combinatorio perché il primo è *ciclico*: le uscite Q e $\overline{Q}$ dipendono l'una dall'altra. Si studiano i due casi $Q = 0$ e $Q = 1$.

- **Caso $Q = 0$.** I_2 riceve in ingresso 0 e produce 1 su $\overline{Q}$; I_1 riceve 1 da $\overline{Q}$ e produce 0 su Q. Lo stato è stabile perché viene confermata la premessa iniziale ($Q = 0$).
- **Caso $Q = 1$.** I_2 riceve 1 e produce 0 su $\overline{Q}$; I_1 riceve 0 e produce 1 su Q. Anche questo stato è stabile.

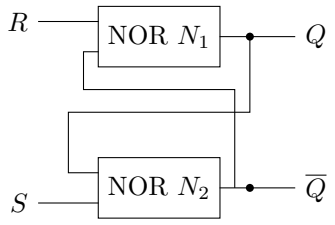
Osservazione. Il circuito ha in realtà anche un terzo possibile stato, con entrambe le uscite circa a metà tra 0 e 1: è detto stato *metastabile*. Inoltre, la variabile di stato è una sola: $\overline{Q}$ non contiene informazione aggiuntiva, essendo il complemento di Q.

Limiti. Due negatori collegati a croce memorizzano un bit, e un insieme di coppie di negatori potrebbe codificare le variabili di stato di una macchina a stati finiti; tuttavia l'utente non ha a disposizione *alcun ingresso* per controllare tali variabili. I latch e i flip-flop che seguono sono provvisti di ingressi per controllare il valore delle variabili di stato, e sono quindi più utili per codificare gli stati di una FSM.

8.2 Latch

8.2.1 Latch SR

Il latch SR si realizza con due porte NOR N_1, N_2 collegate a croce. Ha due ingressi S (*Set*) e R (*Reset*) e due uscite Q e $\overline{Q}$.



Analisi.

- **$S = 0$, $R = 0$.** Nessun ingresso è attivo: l'uscita Q ricorda il suo valore precedente Q_{prec} (e $\overline{Q}$ il suo complemento). Vi sono due stati stabili, a seconda di Q_{prec}.

- $S = 1$, $R = 0$. N_2 riceve almeno un valore vero ($S = 1$) e produce 0 su $\overline{Q}$; a questo punto N_1 ha entrambi gli ingressi a 0 e produce $Q = 1$. Quando viene attivato S, Q viene *settato* a 1.
- $S = 0$, $R = 1$. N_1 riceve almeno un valore vero ($R = 1$) e produce 0 su Q; N_2 ha entrambi gli ingressi a 0 e produce $\overline{Q} = 1$. Quando viene attivato R, Q viene *resettato* a 0.
- $S = 1$, $R = 1$. N_1 e N_2 vedono entrambe almeno un ingresso a 1 e producono entrambe 0. Non ha senso: significa settare e resettare contemporaneamente, e Q e $\overline{Q}$ non sono l'una l'opposto dell'altra (inconsistenti).

S	R	Q	$\overline{Q}$
0	0	Q_{prec}	$\overline{Q}_{\text{prec}}$
0	1	0	1
1	0	1	0
1	1	0	0

Limiti. Il latch SR è scomodo per due motivi:

- si comporta in modo inconsistente quando S e R sono attivati contemporaneamente (entrambe le uscite valgono 0);
- gli ingressi S e R combinano gli aspetti del *come* e del *quando*: attivare uno di essi determina non solo come debba diventare lo stato Q, ma anche quando debba cambiare.

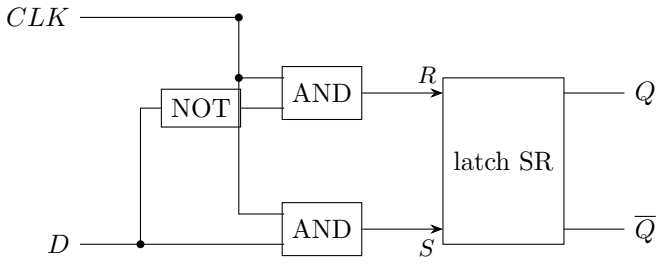
Questi limiti sono superati dal latch D.

8.2.2 Latch D

Il latch D ha due ingressi:

- un ingresso dati D, che controlla il *prossimo stato*;
- un ingresso di clock CLK, che controlla il *momento* del cambio di stato.

Si ottiene da un latch SR ponendo $S = D \cdot CLK$ e $R = \overline{D} \cdot CLK$.



Analisi.

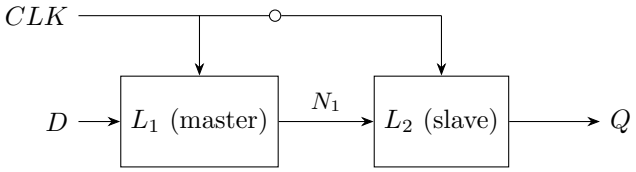
CLK	D	$\overline{D}$	S	R	Q	$\overline{Q}$
0	$\times$	$\overline{\times}$	0	0	Q_{prec}	$\overline{Q}_{\text{prec}}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

Quando $CLK = 0$ il latch ricorda lo stato precedente; quando $CLK = 1$ copia D su Q. Il caso $S = R = 1$ non può mai presentarsi, perché S e R sono sempre uno il complemento dell'altro quando $CLK = 1$.

Limiti. Il latch D aggiorna *continuamente* il suo stato finché $CLK = 1$, mentre sarebbe utile aggiornare lo stato solo in un momento preciso. Questo limite è superato dai flip-flop.

8.3 Flip-flop D

Un flip-flop D si realizza con due latch D in cascata, controllati da due segnali di clock complementari: il primo latch L_1 è detto *master*, il secondo L_2 *slave*; il nodo N_1 collega i due latch.



Analisi.

- Quando $CLK = 0$, il master è trasparente e qualsiasi valore di D viene portato in N_1; lo slave è opaco e mantiene l'uscita.
- Quando CLK diventa 1, il master si blocca e il valore di N_1 viene trasmesso sull'uscita dell'elemento (Q in L_2).

In altre parole, un flip-flop D *copia D su Q al fronte di salita del clock* e ricorda il suo stato in tutti gli altri casi.

Il simbolo circuitale del flip-flop D è un rettangolo con ingressi D e CLK (quest'ultimo contrassegnato da un triangolo, che indica la sensibilità al fronte) e uscite Q e $\overline{Q}$; esiste una versione semplificata con la sola uscita Q.

8.4 Registro di memoria

Definizione. [Registro a N bit] Un *registro* a N bit è un banco di N flip-flop che condividono un ingresso di clock CLK in comune, in modo che tutti i bit vengano aggiornati allo stesso tempo.

8.5 Sintesi di un circuito sequenziale: esempio

Esempio (Semaforo di un incrocio)**. Specifiche.**

- Incrocio tra due strade, nord-sud (NS) ed est-ovest (EO), controllate da un semaforo.
- Il semaforo può commutare ogni 30 secondi; per semplicità si considerano solo rosso e verde.
- Un sensore legge, per ogni direttrice, se esiste almeno un'auto in attesa oppure un'auto che si accinga ad attraversare (condizioni trattate allo stesso modo).
- Il semaforo deve cambiare colore, da rosso a verde, quando esiste un'auto in attesa sulla sua direttrice.
- Se ci sono auto in attesa su entrambe le direttrici il semaforo deve cambiare colore (al termine del tempo di commutazione).

Codifica. Il primo passo è modellare il circuito con una macchina a stati finiti.

- Gli ingressi sono codificati con due variabili booleane i_{EO}, i_{NS}.

- Serve 1 bit per codificare lo stato (due stati possibili): serve dunque un solo bistabile, un flip-flop D F con uscite Q_F, $\overline{Q}_F$. Si pone $Q_F = 1$ per lo stato VerdeNS e $Q_F = 0$ per VerdeEO.
- Serve 1 bit per l'uscita: $u = 1$ codifica LuceVerdeNS, $u = 0$ codifica LuceVerdeEO.

Tabella di transizione. D_F è lo stato futuro.

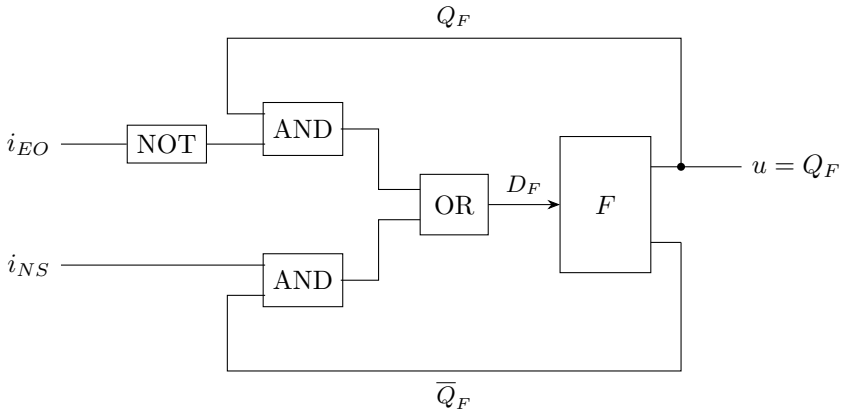
Q_F	i_{EO}	i_{NS}	D_F	u
1	0	0	1	1
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	1	0

Mappe di Karnaugh. Dalla tabella si ottengono

$$D_F = \overline{Q}_F\, i_{NS} + Q_F\, \overline{i_{EO}}, \qquad u = Q_F.$$

Infatti, per $Q_F = 0$ lo stato futuro vale 1 solo se $i_{NS} = 1$ (cioè $D_F = i_{NS}$), mentre per $Q_F = 1$ vale 1 solo se $i_{EO} = 0$ (cioè $D_F = \overline{i_{EO}}$). L'uscita coincide con lo stato.

Circuito. Il circuito si sintetizza con due porte AND, una porta OR e un negatore: il flip-flop D F ha in ingresso $D_F = \overline{Q}_F\, i_{NS} + Q_F\, \overline{i_{EO}}$ e l'uscita u è Q_F.



Osservazione. In sintesi, il procedimento per realizzare un circuito sequenziale è: modellare il problema con una FSM; codificare ingressi, stati e uscite con variabili booleane; scrivere la tabella di transizione (stato futuro e uscita in funzione di stato presente e ingressi); semplificare con le mappe di Karnaugh; realizzare la rete combinatoria e collegarla a un registro di flip-flop D che memorizza lo stato.

Riferimenti. Capitolo 3 del libro di supporto (S. Harris et al., *Sistemi Digitali e Architettura dei Calcolatori*), sezioni 3.1, 3.2, 3.2.1, 3.2.2, 3.2.3 e 3.2.4.